

PlateSense

How the Application Works

Technical overview and system documentation for the current PlateSense prototype

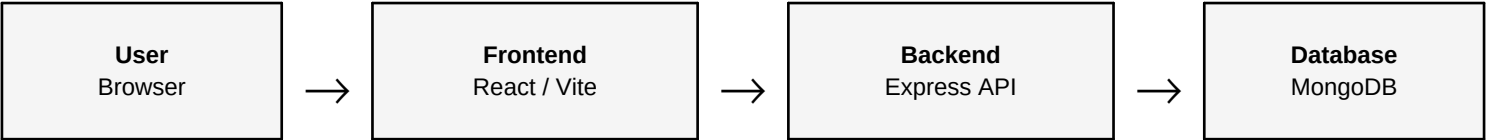
Frontend React + Vite + JavaScript	Backend Node.js + Express	Database MongoDB + Mongoose
Authentication JWT + bcryptjs	Core function Calorie tracking	Configuration Environment variables

This document describes the current application structure, authentication flow, calorie-tracking flow, API layer, database interaction, and development status.

1. Overview

PlateSense is a web-based calorie tracking application. Its main purpose is to allow a user to create an account, sign in, record calories consumed during the day, view a daily calorie total, and set a personal daily calorie target.

System Architecture



The frontend communicates with the Express backend using HTTP API requests. The backend handles application logic and communicates with MongoDB through Mongoose.

2. Technology Stack

Layer	Technology	Role
Frontend	React / Vite / JavaScript	User interface and API calls
Backend	Node.js / Express	HTTP API and application logic
Database	MongoDB	Persistent application data
ODM	Mongoose	MongoDB data access
Authentication	JWT	Authenticated API access
Password security	bcryptjs	Password hashing and comparison
Development	Nodemon	Automatic backend restart during development

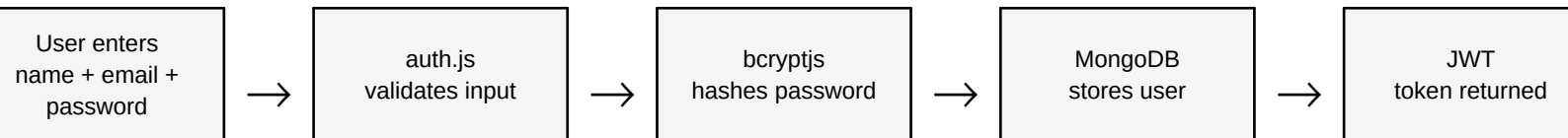
3. Project Structure

```
platesense/  
  ■■■ src/  
  ■ ■■■ App.jsx  
  ■■■ public/  
  ■■■ style.css  
  ■■■ package.json  
  ■■■ server/  
  ■■■ server.js  
  ■■■ .env  
  ■■■ models/  
  ■ ■■■ User.js  
  ■■■ routes/  
  ■■■ auth.js  
  ■■■ users.js  
  ■■■ calories.js
```

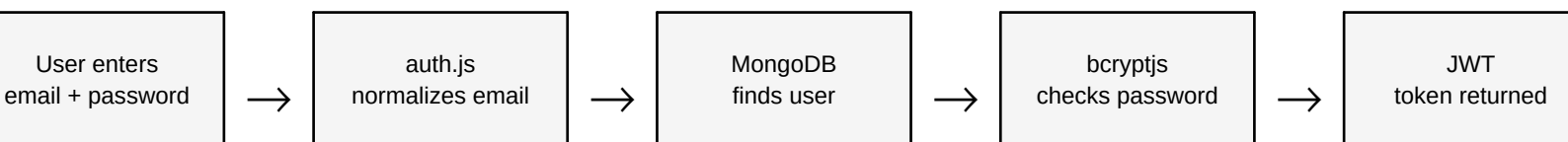
4. Authentication

PlateSense uses account-based authentication. During registration, the password is hashed before it is stored. During login, bcryptjs compares the supplied password with the stored hash. A successful login produces a JWT token.

Registration Flow



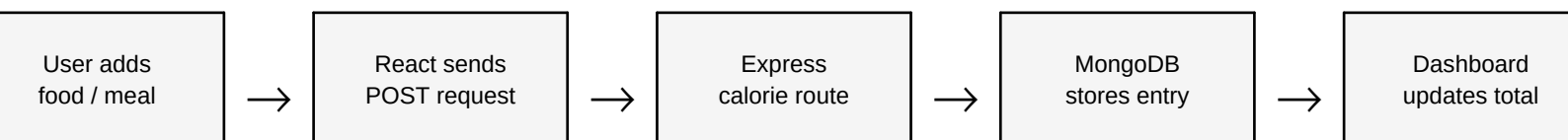
Login Flow



The current authentication implementation normalizes email addresses with `trim()` and `lowercase()` during registration and login. The frontend stores the returned token in `localStorage` using the key `platesenseToken`.

5. Calorie Tracking Flow

Once authenticated, the user can add food or meals and their calorie amounts. The backend stores the entries and the dashboard retrieves today's entries and displays the daily total.

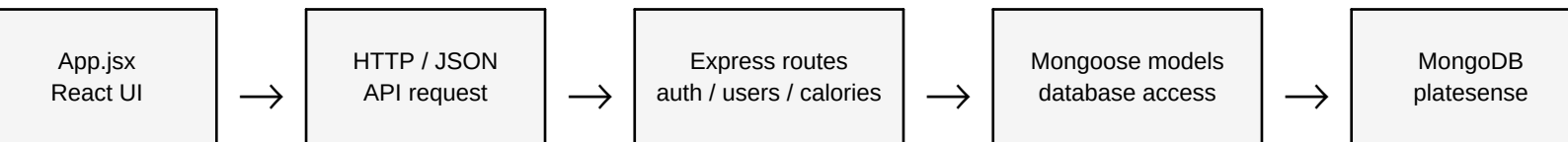


- Individual calorie entries can be deleted.
- Today's entries can be reset.
- The dashboard can compare the consumed total with the user's daily target.

6. API Layer

Method	Endpoint	Purpose
POST	/api/auth/register	Create a new user account.
POST	/api/auth/login	Authenticate an existing user.
GET	/api/users/me	Retrieve the authenticated user's information.
GET	/api/calories/today	Retrieve today's calorie entries.
POST	/api/calories	Add a calorie/food entry.
DELETE	/api/calories/:id	Delete a specific calorie entry.
DELETE	/api/calories/today/reset	Remove today's calorie entries.
PUT	/api/users/target	Update the user's daily calorie target.

7. Frontend → API → Database Relationship



The frontend does not communicate directly with MongoDB. Requests pass through the Express backend, which provides the application boundary and handles authentication and data operations.

8. MongoDB Configuration

The development configuration uses a local MongoDB database named platesense. The backend uses the MONGO_URI environment variable to connect to the database.

```
mongodb://127.0.0.1:27017/platesense
```

Passwords should be stored as bcrypt hashes rather than plain-text passwords.

9. Typical User Journey

1. Open PlateSense.
2. Register an account or log in.
3. Receive and store the authentication token.
4. Load the user's information and today's calorie entries.
5. Add food or meals with calorie amounts.
6. Review the day's calorie total.
7. Compare the total with the daily calorie target.
8. Delete individual entries or reset today's entries if necessary.
9. Log out when finished.

10. Login Troubleshooting

If login displays "Invalid email or password", the backend uses the same response for an unknown email and an incorrect password. This avoids revealing whether a particular email address has an account.

- Check that the account exists in the platesense database.
- Check that the stored email is normalized, for example test4@example.com.
- Check that the account was created against the same MongoDB database used by the running backend.
- Use the browser Network tab to inspect the /auth/login request and HTTP status.
- Check the backend terminal for errors.

11. Current Development Status

The project currently includes the main frontend, Express backend, MongoDB connection, JWT-based authentication, user routes, calorie routes, and daily target functionality in development. The authentication and login flow is currently being tested and refined.

12. Future Development Areas

- Improve authentication and account-management UX.
- Add stronger validation and clearer user feedback.
- Add more detailed food and meal information.
- Improve calorie history and reporting.
- Add additional account settings.
- Add automated frontend and backend tests.
- Prepare production configuration and secure deployment.

13. Security Notes

- Never store user passwords as plain text.
- Keep JWT_SECRET private.
- Do not commit .env files containing secrets to source control.
- Use HTTPS when deploying publicly.
- Validate and authorize authenticated API requests on the backend.